

AeroFit Trademill - Data Analysis

Suhel Mandal

```
In [ ]: !jupyter nbconvert -- to html
```

1. Importing Libraries and Loading the Dataset

```
In [ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [ ]: from google.colab import files
files = files.upload()
```

Choose Files no files selected Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving aerofit_treadmill.csv to aerofit_treadmill.csv

```
In [ ]: df = pd.read_csv("/content/aerofit_treadmill.csv")
df
```

Out[]:

	Product	Age	Gender	Education	MaritalStatus	Usage	Fitness	Income	Miles
0	KP281	18	Male	14	Single	3	4	29562	112
1	KP281	19	Male	15	Single	2	3	31836	75
2	KP281	19	Female	14	Partnered	4	3	30699	66
3	KP281	19	Male	12	Single	3	3	32973	85
4	KP281	20	Male	13	Partnered	4	2	35247	47
...	...	...	...	...	...	...	...	...	...
175	KP781	40	Male	21	Single	6	5	83416	200
176	KP781	42	Male	18	Single	5	4	89641	200
177	KP781	45	Male	16	Single	5	5	90886	160
178	KP781	47	Male	18	Partnered	4	5	104581	120
179	KP781	48	Male	18	Partnered	4	5	95508	180

180 rows × 9 columns

```
In [ ]: print(f'  Rows    : {df.shape[0]}')
print(f'  Columns : {df.shape[1]}')

Rows    : 180
Columns : 9
```

Goal: Analyzing Netflix data to help the Business Decide:

- 1. To identify the right customer for each type of product
- 2. Increase revenue by upselling
- 3. To understand the Gender and Marital status Gaps
- 4. Predicting the probability of purchase
- 5. Identify customer segments and optimise store inventory

```
In [ ]: print('Data Types')
df.dtypes
```

Data Types

Out[]: 0

Product	object
Age	int64
Gender	object
Education	int64
MaritalStatus	object
Usage	int64
Fitness	int64
Income	int64
Miles	int64

dtype: object

In []: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 180 entries, 0 to 179
Data columns (total 9 columns):
Column Non-Null Count Dtype
--- ---
0 Product 180 non-null object
1 Age 180 non-null int64
2 Gender 180 non-null object
3 Education 180 non-null int64
4 MaritalStatus 180 non-null object
5 Usage 180 non-null int64
6 Fitness 180 non-null int64
7 Income 180 non-null int64
8 Miles 180 non-null int64
dtypes: int64(6), object(3)
memory usage: 12.8+ KB

In []: df.describe(include = 'all')

	Product	Age	Gender	Education	MaritalStatus	Usage	Fitness	Income	Miles
count	180	180.000000	180	180.000000	180	180.000000	180.000000	180.000000	180.000000
unique	3	NaN	2	NaN	2	NaN	NaN	NaN	NaN
top	KP281	NaN	Male	NaN	Partnered	NaN	NaN	NaN	NaN
freq	80	NaN	104	NaN	107	NaN	NaN	NaN	NaN
mean	NaN	28.788889	NaN	15.572222	NaN	3.455556	3.311111	53719.577778	103.194444
std	NaN	6.943498	NaN	1.617055	NaN	1.084797	0.958869	16506.684226	51.863605
min	NaN	18.000000	NaN	12.000000	NaN	2.000000	1.000000	29562.000000	21.000000
25%	NaN	24.000000	NaN	14.000000	NaN	3.000000	3.000000	44058.750000	66.000000
50%	NaN	26.000000	NaN	16.000000	NaN	3.000000	3.000000	50596.500000	94.000000
75%	NaN	33.000000	NaN	16.000000	NaN	4.000000	4.000000	58668.000000	114.750000
max	NaN	50.000000	NaN	21.000000	NaN	7.000000	5.000000	104581.000000	360.000000

Checking for missing values

In []: missing_value_counts = df.isna().sum()
missing_value_perc = missing_value_counts / len(df) * 100

missing_value_table = pd.DataFrame({'Missing Value Count': missing_value_counts, 'Missing Value Percentage': missing_value_perc})

Out []:

	Missing Value Count	Missing Value Percentage
Product	0	0.0
Age	0	0.0
Gender	0	0.0
Education	0	0.0
MaritalStatus	0	0.0
Usage	0	0.0
Fitness	0	0.0
Income	0	0.0
Miles	0	0.0

Converting Categorical COLUMNS

In []:

```
df['Product'] = df['Product'].astype('category')
df['Gender'] = df['Gender'].astype('category')
df['MaritalStatus'] = df['MaritalStatus'].astype('category')

print('The required columns have been converted to Category')
df[['Product', 'Gender', 'MaritalStatus']].dtypes
```

The required columns have been converted to Category

Out []:

	0
Product	category
Gender	category
MaritalStatus	category

dtype: object

2. Non-Graphical Analysis

In []:

```
print('The count of Product, Gender & Marital Status is displayed below:')
print('-----Product Count-----')
print(df['Product'].value_counts())
print('\n-----Gender Count-----')
print(df['Gender'].value_counts())
print('\n-----Marital Status Count-----')
print(df['MaritalStatus'].value_counts())
```

The count of Product, Gender & Marital Status is displayed below:

-----Product Count-----

Product	
KP281	80
KP481	60
KP781	40

Name: count, dtype: int64

-----Gender Count-----

Gender	
Male	104
Female	76

Name: count, dtype: int64

-----Marital Status Count-----

MaritalStatus	
Partnered	107
Single	73

Name: count, dtype: int64

3. Outlier Detection

Outliers for every continuous variable in the dataset

In []:

```
# ----- Data without clipping -----
```

```

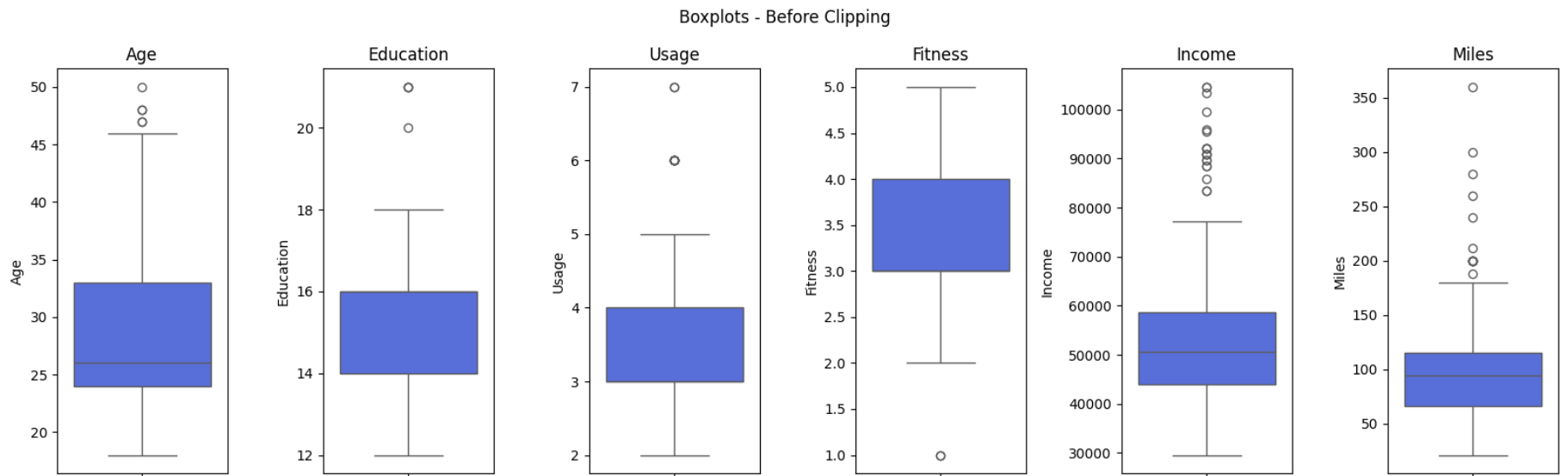
cont_cols = ['Age', 'Education', 'Usage', 'Fitness', 'Income', 'Miles']

fig, axes = plt.subplots(1,6, figsize=(16,5))

for i,col in enumerate(cont_cols):
    sns.boxplot(y = df[col], ax = axes[i], color='#4361ee')
    axes[i].set_title(col)

plt.suptitle('Boxplots - Before Clipping')
plt.tight_layout()
plt.show()

```



```

In [ ]: # ----- Data with 5 percentile and 95 percentile clipping -----

df_clipped = df.copy()

for col in cont_cols:
    low_perc = np.percentile(df[col],5)
    high_perc = np.percentile(df[col],95)

    df_clipped[col] = np.clip(df[col],low_perc,high_perc)

print('The required columns have been clipped')
print('-----Comparing the Income Values-----')
print(f'Maximum Income before the data clipping:{df["Income"].max()}')
print(f'Maximum Income after the data clipping:{df_clipped["Income"].max()}')

```

The required columns have been clipped
-----Comparing the Income Values-----
Maximum Income before the data clipping:104581
Maximum Income after the data clipping:90948.24999999999

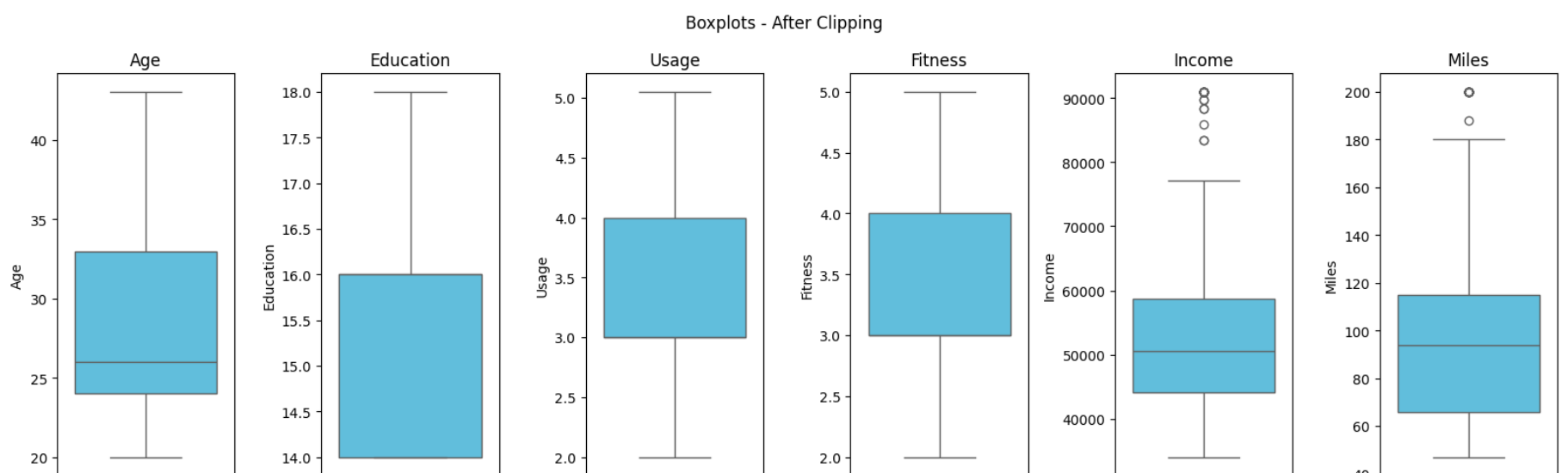
```

In [ ]: fig, axes = plt.subplots(1,6, figsize=(16,5))

for i,col in enumerate(cont_cols):
    sns.boxplot(y=df_clipped[col], ax=axes[i], color='#4cc9f0')
    axes[i].set_title(col)

plt.suptitle('Boxplots - After Clipping')
plt.tight_layout()
plt.show()

```



4. Univariate Analysis

```

In [ ]: # Histogram for all continuous variables of the dataset

```

```

fig, axes = plt.subplots(2,3,figsize =(16,12))
axes = axes.flatten()

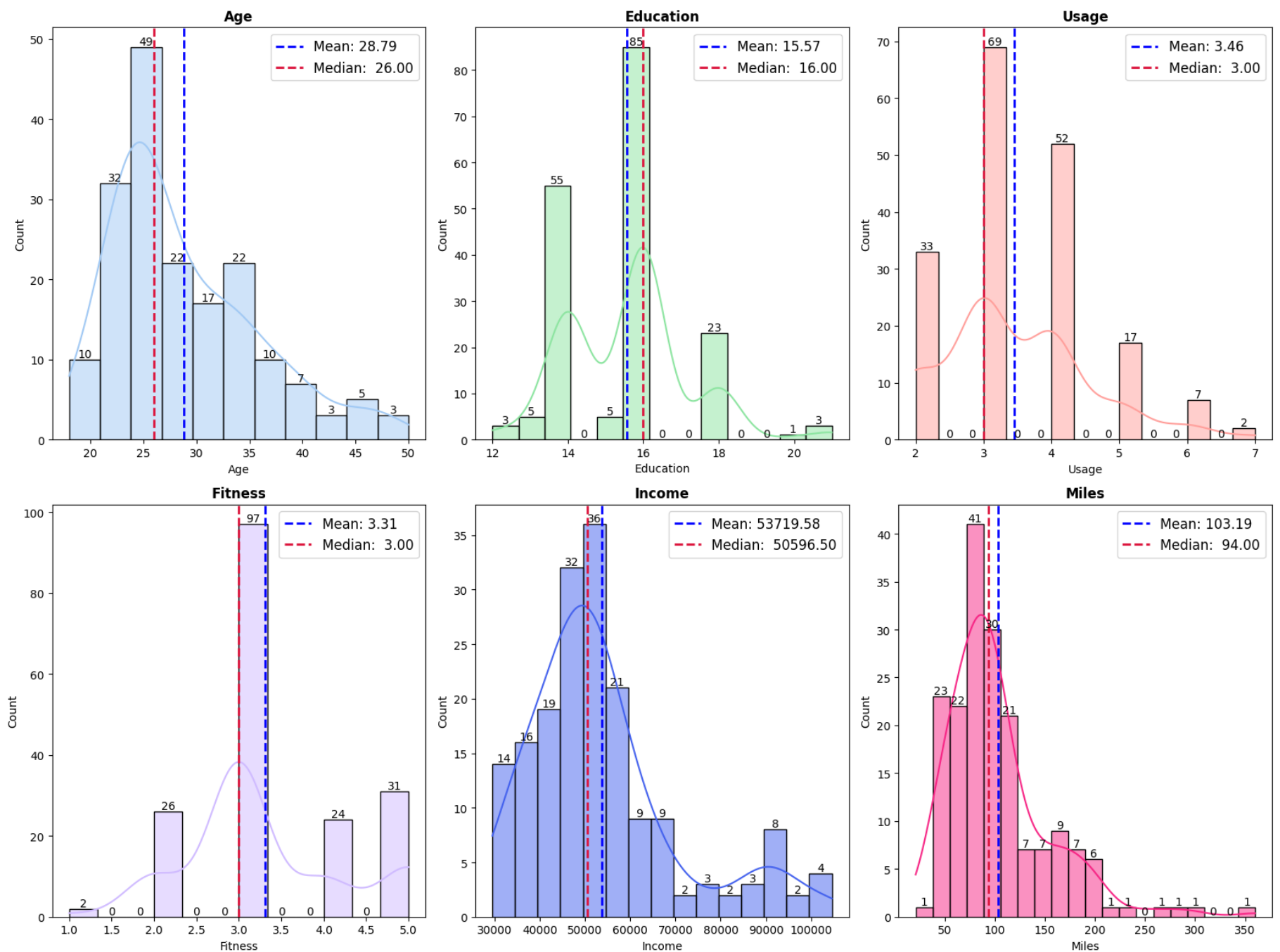
colors = ['#a1c9f4', '#8de5a1', '#ff9f9b', '#d0bbff', '#4361ee', '#f72585']

for i,col in enumerate(cont_cols):
    ax = axes[i]
    sns.histplot(df[col], ax=axes[i], kde=True, color=colors[i]) # Changed scatterplot to histplot and remove
    axes[i].set_title(col)
    mean_value = df[col].mean()
    median_value = df[col].median()
    axes[i].axvline(mean_value, color = 'blue', linestyle = '--', linewidth = 2, label = f'Mean: {mean_value}')
    axes[i].axvline(median_value, color = 'crimson', linestyle = '--', linewidth = 2, label = f'Median: {median_value}')
    axes[i].set_title(col, fontweight = 'bold')
    axes[i].legend(fontsize = 12)

    for i in ax.containers:
        ax.bar_label(i)

plt.tight_layout()
plt.show()

```



```

In [ ]: from matplotlib.figure import Figure
# Countplot for categorical columns of the dataset

cat_cols=['Product', 'Gender', 'MaritalStatus']

fig, axes= plt.subplots(1,3, figsize = (14,6))

for i,col in enumerate(cat_cols):
    ax = axes[i]
    sns.countplot(data = df, x = col, ax = axes[i], palette = 'ch:s=.25,rot=-.25', edgecolor = 'darkblue')
    axes[i].set_title(f'{col} -- Value Counts', fontweight='bold')
    axes[i].legend(fontsize = 10)

    for i in ax.containers:
        ax.bar_label(i)

plt.legend()
plt.tight_layout()
plt.show()

```

```
/tmp/ipykernel_524/3101314303.py:10: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(data = df, x = col, ax = axes[i], palette = 'ch:s=.25,rot=-.25', edgecolor = 'darkblue')
/tmp/ipykernel_524/3101314303.py:12: UserWarning: No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.
axes[i].legend(fontsize = 10)
```

```
/tmp/ipykernel_524/3101314303.py:10: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

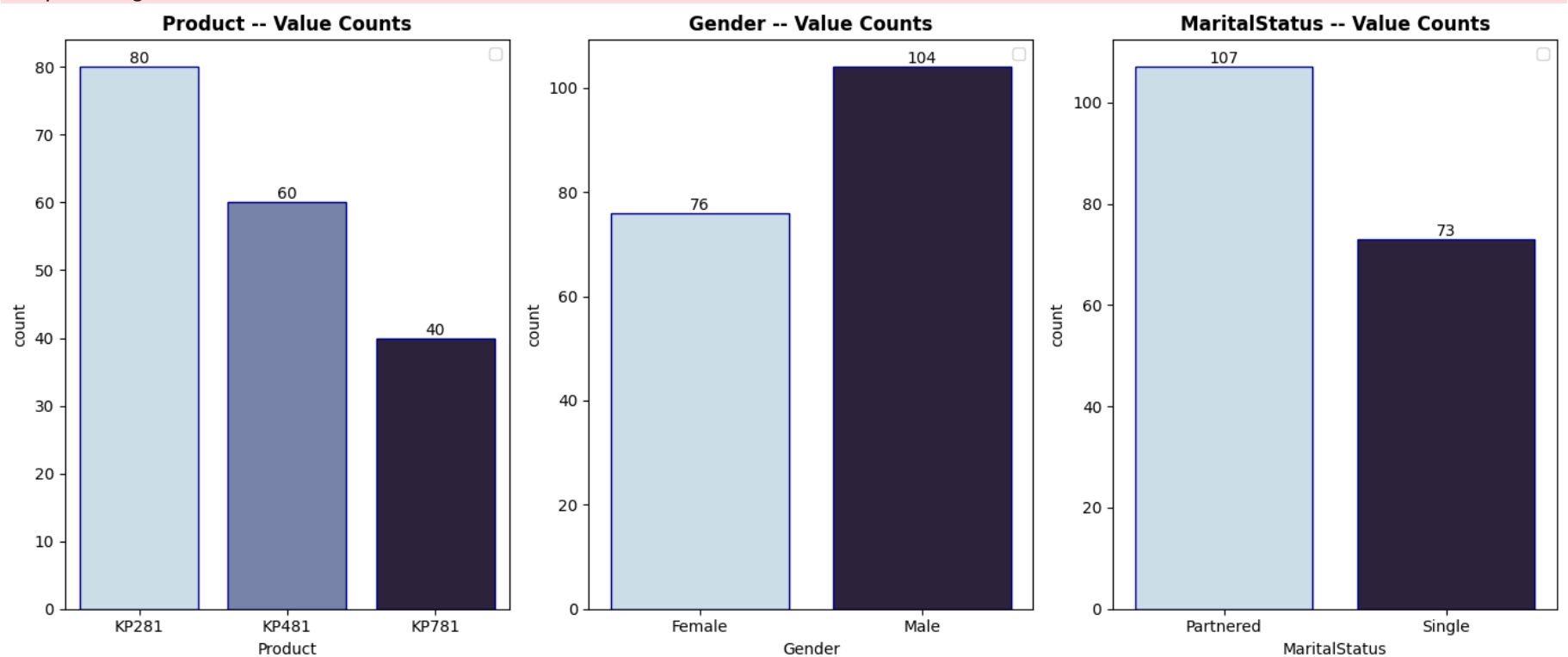
```
sns.countplot(data = df, x = col, ax = axes[i], palette = 'ch:s=.25,rot=-.25', edgecolor = 'darkblue')
/tmp/ipykernel_524/3101314303.py:12: UserWarning: No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.
axes[i].legend(fontsize = 10)
```

```
/tmp/ipykernel_524/3101314303.py:10: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(data = df, x = col, ax = axes[i], palette = 'ch:s=.25,rot=-.25', edgecolor = 'darkblue')
/tmp/ipykernel_524/3101314303.py:12: UserWarning: No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.
axes[i].legend(fontsize = 10)
```

```
/tmp/ipykernel_524/3101314303.py:17: UserWarning: No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.
plt.legend()
```



5. Bivariate Analysis

```
In [ ]: fig, axes = plt.subplots(1,3,figsize = (12,6))

sns.countplot(data = df, x = 'Gender', hue = 'Product',ax = axes[0], palette = 'blend:#7AB,#EDA', edgecolor = 'darkblue')
axes[0].set_title('Gender vs Product Purchased', fontweight = 'bold')

for i in axes[0].containers:
    axes[0].bar_label(i)

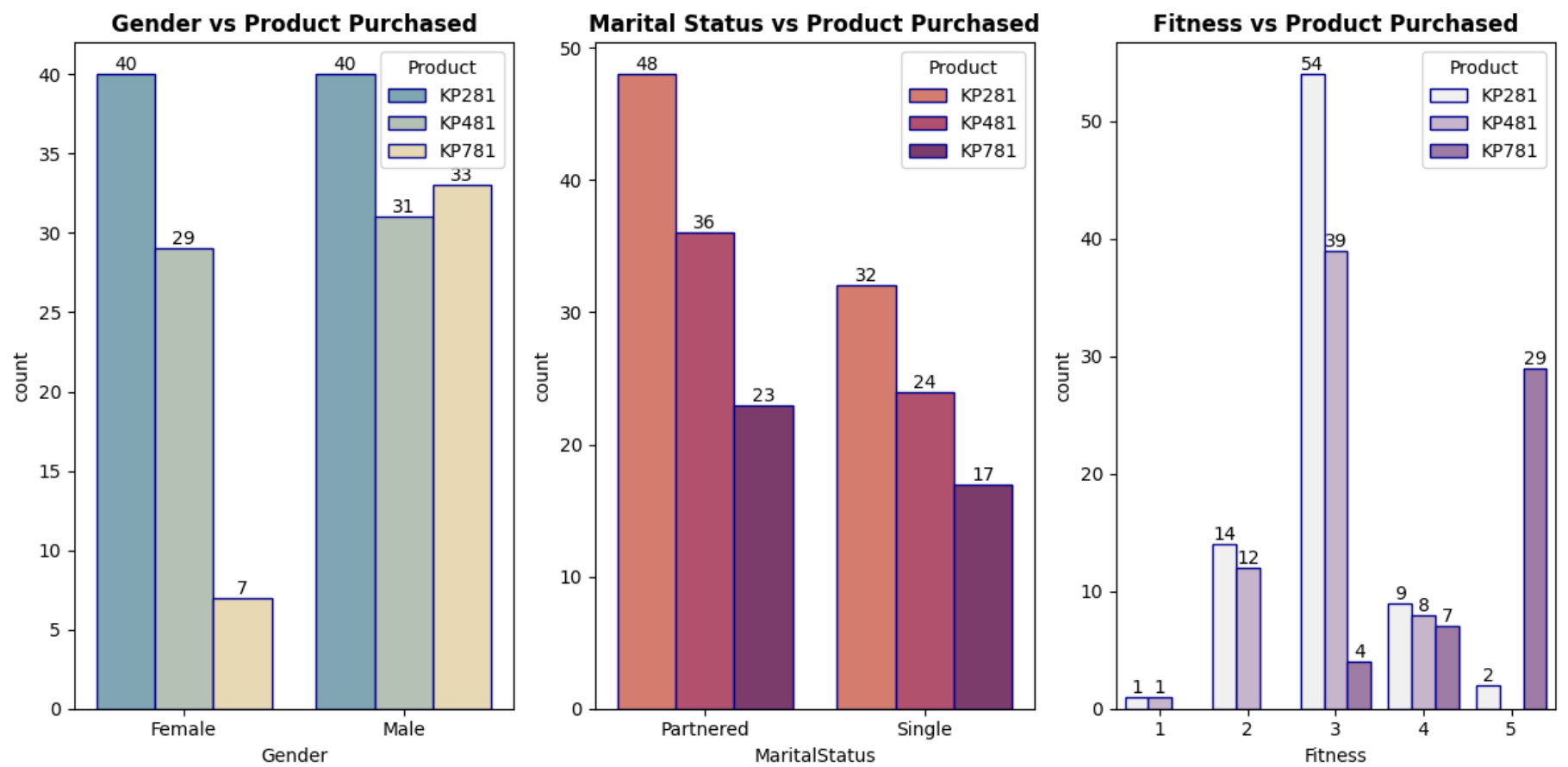
sns.countplot(data = df, x = 'MaritalStatus', hue = 'Product',ax = axes[1], palette = 'flare', edgecolor = 'darkblue')
axes[1].set_title('Marital Status vs Product Purchased', fontweight = 'bold')

for j in axes[1].containers:
    axes[1].bar_label(j)

custom_palette_fitness =sns.light_palette('#a275ac', n_colors=3)
sns.countplot(data = df, x = 'Fitness', hue = 'Product',ax = axes[2], palette = custom_palette_fitness, edgecolor = 'darkblue')
axes[2].set_title('Fitness vs Product Purchased', fontweight = 'bold')

for k in axes[2].containers:
    axes[2].bar_label(k)

plt.tight_layout()
plt.show()
```

```
In [ ]: fig, axes = plt.subplots(2,3,figsize = (14, 11))
axes = axes.flatten()

for i,col in enumerate(cont_cols):
    sns.boxplot(data = df, x = 'Product', y = col, ax=axes[i], order=['KP281', 'KP481', 'KP781'], palette = '
    axes[i].set_title(f'{col} vs Product Graph')

plt.suptitle('Numeric Variables vs Product Comparison - Boxplots', fontsize=15)
plt.tight_layout()
plt.show()
```

/tmp/ipykernel_524/1753074126.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(data = df, x = 'Product', y = col, ax=axes[i], order=['KP281', 'KP481', 'KP781'], palette = '
light:#5A9')
```

/tmp/ipykernel_524/1753074126.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(data = df, x = 'Product', y = col, ax=axes[i], order=['KP281', 'KP481', 'KP781'], palette = '
light:#5A9')
```

/tmp/ipykernel_524/1753074126.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(data = df, x = 'Product', y = col, ax=axes[i], order=['KP281', 'KP481', 'KP781'], palette = '
light:#5A9')
```

/tmp/ipykernel_524/1753074126.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(data = df, x = 'Product', y = col, ax=axes[i], order=['KP281', 'KP481', 'KP781'], palette = '
light:#5A9')
```

/tmp/ipykernel_524/1753074126.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

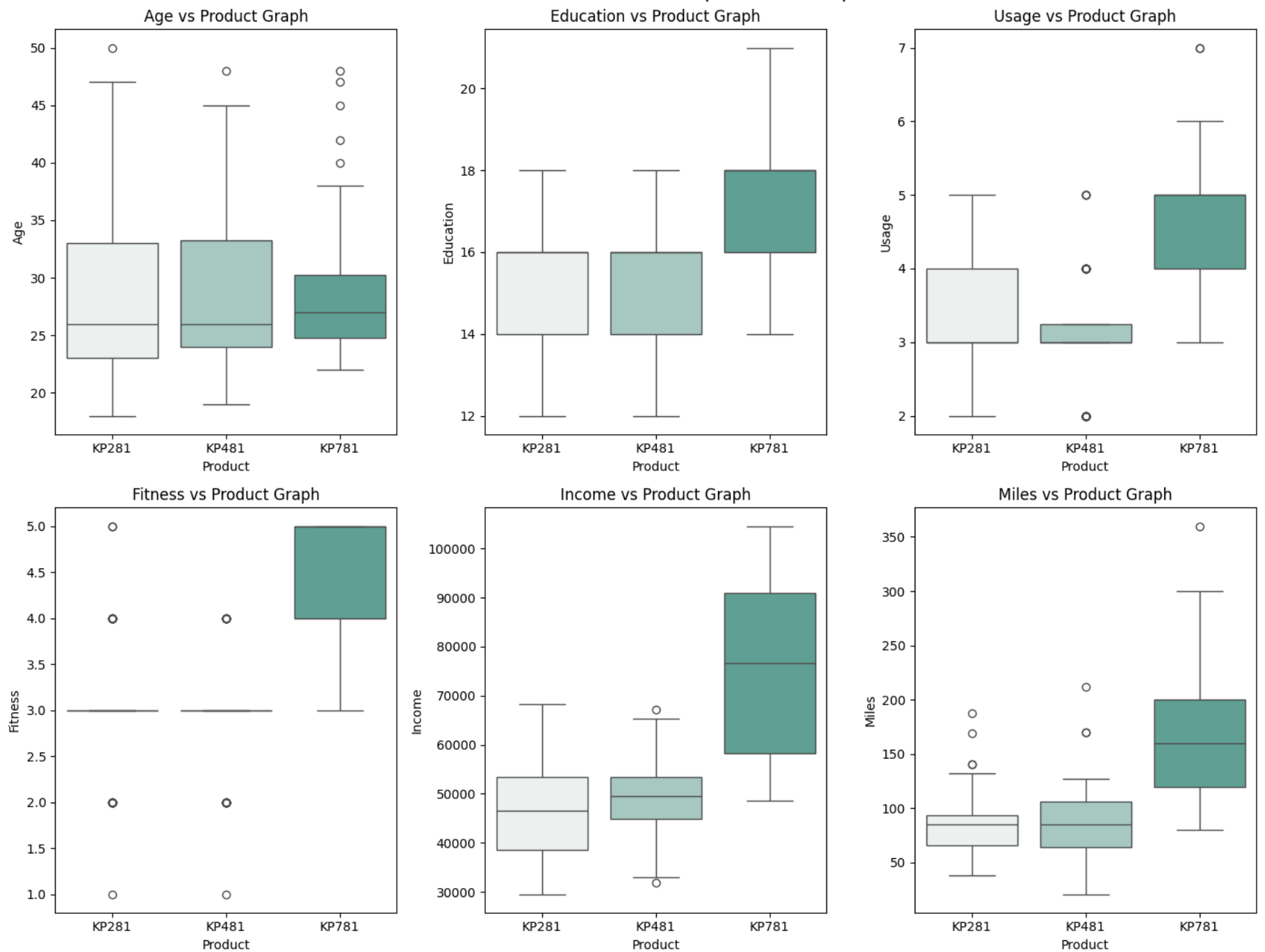
```
sns.boxplot(data = df, x = 'Product', y = col, ax=axes[i], order=['KP281', 'KP481', 'KP781'], palette = '
light:#5A9')
```

/tmp/ipykernel_524/1753074126.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(data = df, x = 'Product', y = col, ax=axes[i], order=['KP281', 'KP481', 'KP781'], palette = '
light:#5A9')
```


Numeric Variables vs Product Comparison — Boxplots



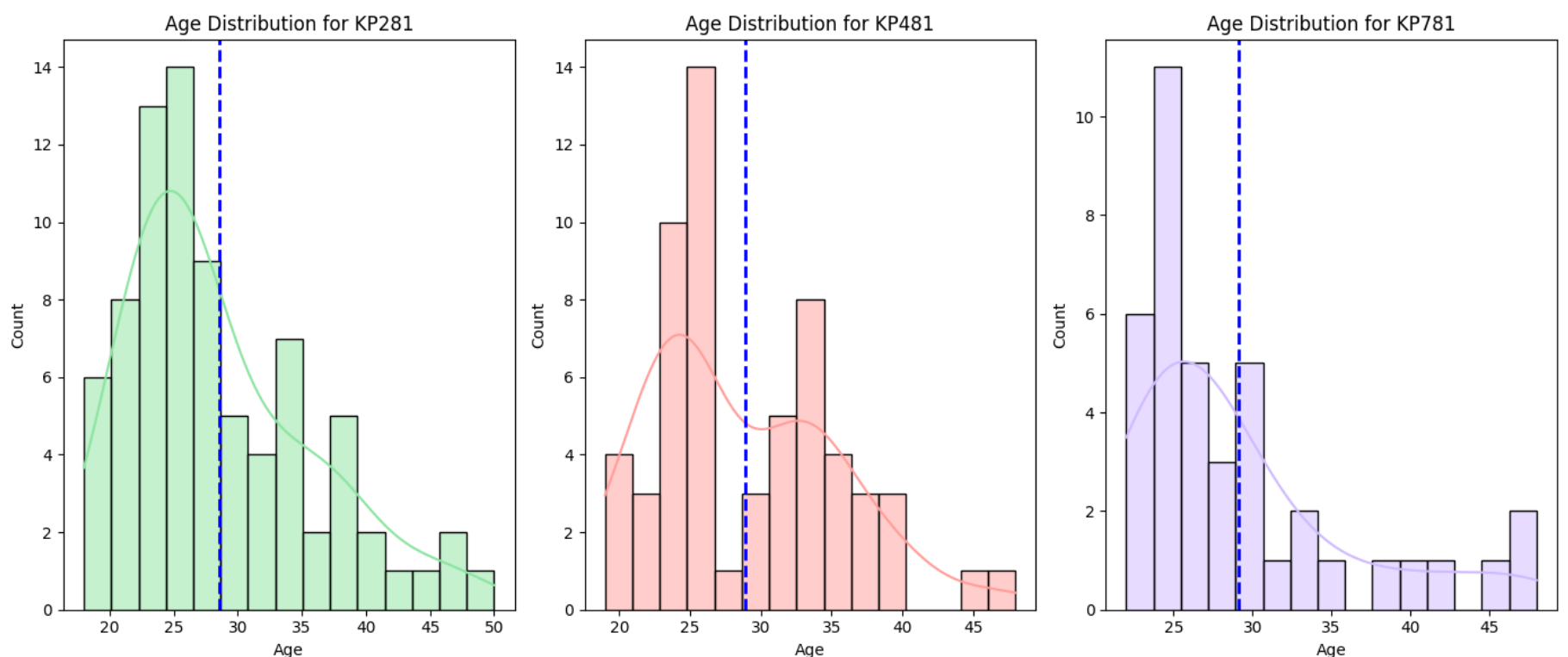
```
In [ ]: fig, axes = plt.subplots(1,3,figsize=(14,6))

models = ['KP281', 'KP481', 'KP781']
colour = ['#8de5a1', '#ff9f9b', '#d0bbff']

for i, model in enumerate(models):
    filtered_products = df[df['Product']== model]
    sns.histplot(data = filtered_products, x = "Age", kde = True, bins = 15, ax = axes[i], color = colour[i])
    axes[i].axvline(filtered_products['Age'].mean(), color = 'blue', linestyle = '--', linewidth = 2, label=f'Mean: {filtered_products["Age"].mean():.1f}')
    axes[i].set_title(f'Age Distribution for {model}')
    axes[i].set_xlabel('Age')

    #axes[i].axvline(mean_value, color = 'blue', linestyle = '--', linewidth = 2, label = f'Mean: {mean_value:.1f}')

plt.tight_layout()
plt.show()
```



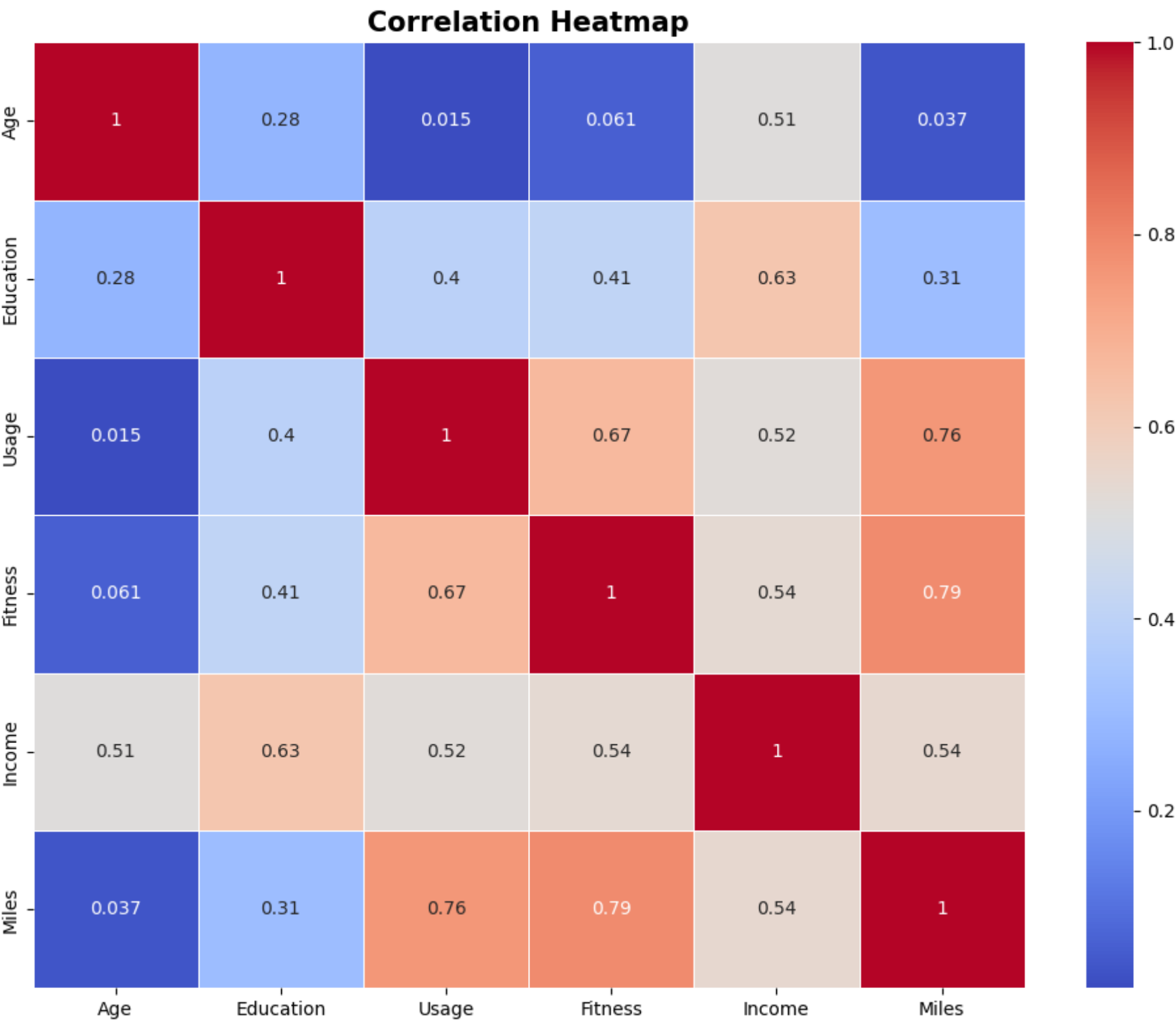
6. Correlation


```
In [ ]: correlation_matrix = df[cont_cols].corr()  
correlation_matrix
```

Out []:

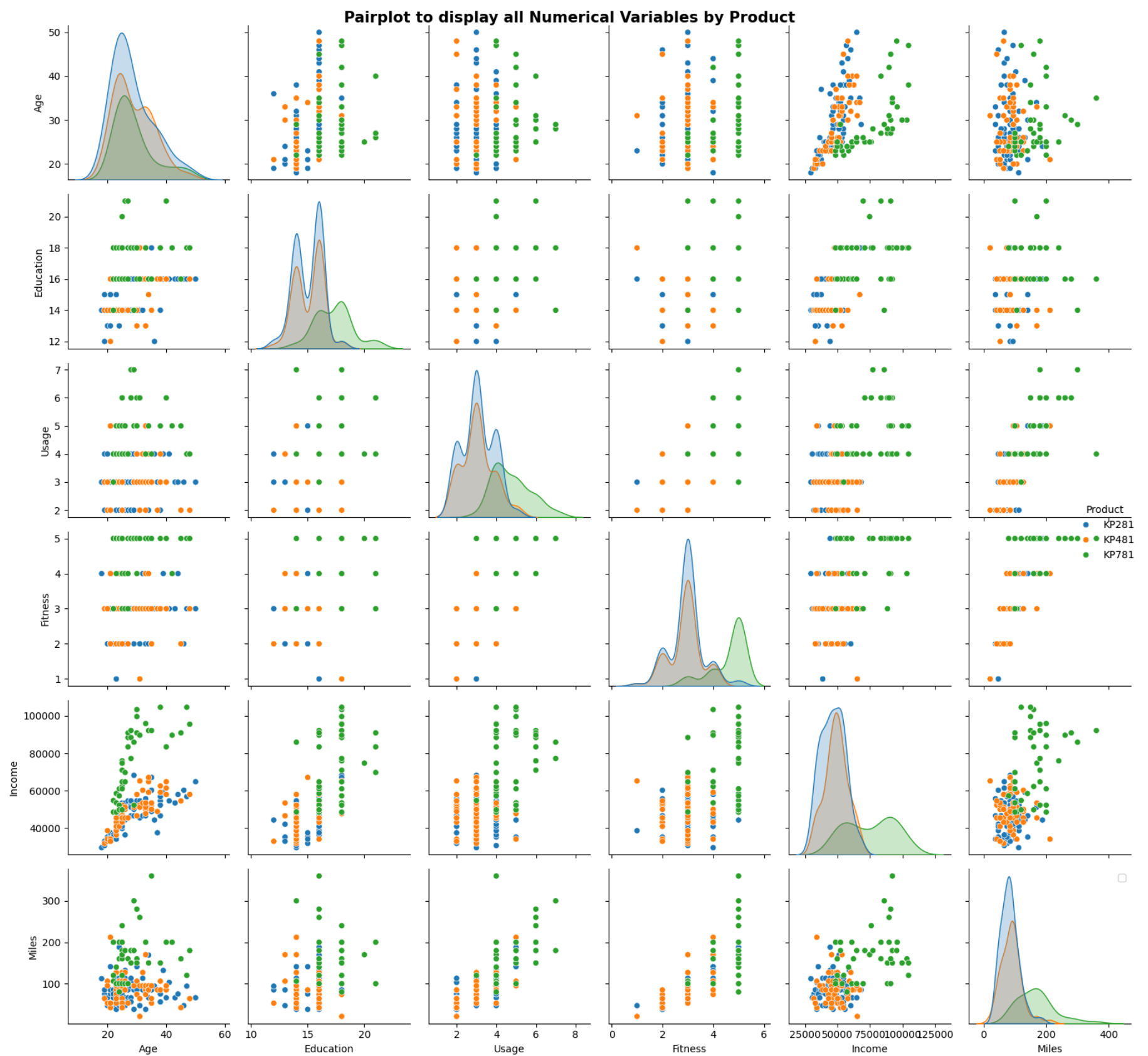
	Age	Education	Usage	Fitness	Income	Miles
Age	1.000000	0.280496	0.015064	0.061105	0.513414	0.036618
Education	0.280496	1.000000	0.395155	0.410581	0.625827	0.307284
Usage	0.015064	0.395155	1.000000	0.668606	0.519537	0.759130
Fitness	0.061105	0.410581	0.668606	1.000000	0.535005	0.785702
Income	0.513414	0.625827	0.519537	0.535005	1.000000	0.543473
Miles	0.036618	0.307284	0.759130	0.785702	0.543473	1.000000

```
In [ ]: plt.figure(figsize=(10,8))  
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', linewidth = 0.5)  
plt.title('Correlation Heatmap', fontsize = 15, fontweight = 'bold')  
plt.tight_layout()  
plt.show()
```



```
In [ ]: sns.pairplot(data = df, vars = ['Age','Education','Usage','Fitness','Income','Miles'], hue = "Product" )  
plt.suptitle('Pairplot to display all Numerical Variables by Product', fontsize = 15, fontweight = 'bold')  
plt.legend()  
plt.tight_layout()  
plt.show()
```

/tmp/ipykernel_524/2992300464.py:3: UserWarning: No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.
plt.legend()



7. Probability Analysis

Marginal Probability

```
In [ ]: marginal_probability = df["Product"].value_counts(normalize = True)
print("The Marginal Probability is displayed below: \n")
print(marginal_probability)

print("The Marginal Probability as percentage: \n")
print((marginal_probability * 100).round(2).astype(str) + '%')
```

The Marginal Probability is displayed below:

```
Product
KP281    0.444444
KP481    0.333333
KP781    0.222222
Name: proportion, dtype: float64
The Marginal Probability as percentage:
```

```
Product
KP281    44.44%
KP481    33.33%
KP781    22.22%
Name: proportion, dtype: object
```

```
In [ ]: plt.figure(figsize=(5,5))

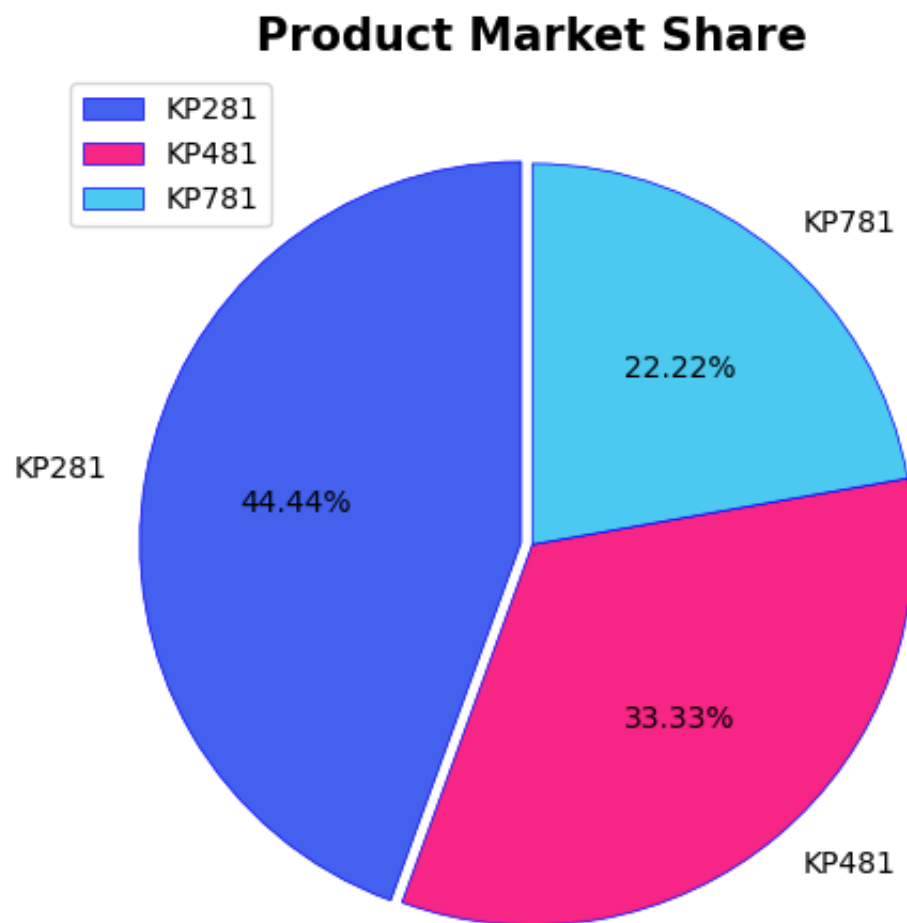
plt.pie(marginal_probability,
        labels = marginal_probability.index,
        startangle= 90,
```

```

        explode = (0.03,0,0),
        autopct = '%.2f%%',
        colors=[ '#4361ee', '#f72585', '#4cc9f0'],
        wedgeprops={'edgecolor': 'blue', 'linewidth': 0.5})

plt.title("Product Market Share", fontsize = 15, fontweight = 'bold')
plt.legend()
plt.tight_layout()
plt.show()

```



Conditional Probability

```

In [ ]: # ----- Conditional Probability – P(Product | Gender) -----

ctab_gender = pd.crosstab(index = df['Gender'], columns = df['Product'], normalize = 'index')
print('P(Product | Gender): \n')
print(ctab_gender)

pro_gender = ctab_gender.loc['Female', 'KP481']
print(f'\nP(KP481 | Female) = {pro_gender} → {pro_gender*100:.1f}% of female customers bought KP481')

```

P(Product | Gender):

Product	KP281	KP481	KP781
Gender			
Female	0.526316	0.381579	0.092105
Male	0.384615	0.298077	0.317308

P(KP481 | Female) = 0.3815789473684211 → 38.2% of female customers bought KP481

```

In [ ]: # ----- Conditional Probability – P(Product | Marital Status) -----

ctab_marital = pd.crosstab(index = df['MaritalStatus'], columns = df['Product'], normalize = True)
print('P(Product | Marital Status): \n')
print(ctab_marital)

pro_marital = ctab_marital.loc['Partnered', 'KP481']
print(f'\nP(KP481 | Partnered) = {pro_marital} → {pro_marital*100:.1f}% of partnered customers bought KP481')

```

P(Product | Marital Status):

Product	KP281	KP481	KP781
MaritalStatus			
Partnered	0.266667	0.200000	0.127778
Single	0.177778	0.133333	0.094444

P(KP481 | Partnered) = 0.2 → 20.0% of partnered customers bought KP481

8. Customer Profiling

```
In [ ]: # ----- Profile for each Product -----

for product in ['KP281', 'KP481', 'KP781']:
    item = df[df['Product'] == product]

    print('-----')
    print(f'Profile: {product}  (n = {len(item)})')
    print('-----')
    print(f'Avg Age      : {item["Age"].mean():.1f}')
    print(f'Avg Income   : ₹{item["Income"].mean():.1f}')
    print(f'Avg Miles    : {item["Miles"].mean():.0f}')
    print(f'Avg Usage    : {item["Usage"].mean():.1f}x')
    print(f'Avg Fitness  : {item["Fitness"].mean():.1f}/5')

    male_pct = (item['Gender'] == 'Male').mean() * 100
    female_pct = (item['Gender'] == 'Female').mean() * 100
    partnered_pct = (item['MaritalStatus'] == 'Partnered').mean() * 100
    single_pct = (item['MaritalStatus'] == 'Single').mean() * 100

    print(f'Male %      : {male_pct:.0f}%')
    print(f'Female %    : {female_pct:.0f}%')
    print(f'Partnered % : {partnered_pct:.0f}%')
    print(f'Single %    : {single_pct:.0f}%')
    print()
```

```
-----
Profile: KP281  (n = 80)
-----
Avg Age      : 28.6
Avg Income   : ₹46418.0
Avg Miles    : 83
Avg Usage    : 3.1x
Avg Fitness  : 3.0/5
Male %      : 50%
Female %    : 50%
Partnered % : 60%
Single %    : 40%
```

```
-----
Profile: KP481  (n = 60)
-----
Avg Age      : 28.9
Avg Income   : ₹48973.7
Avg Miles    : 88
Avg Usage    : 3.1x
Avg Fitness  : 2.9/5
Male %      : 52%
Female %    : 48%
Partnered % : 60%
Single %    : 40%
```

```
-----
Profile: KP781  (n = 40)
-----
Avg Age      : 29.1
Avg Income   : ₹75441.6
Avg Miles    : 167
Avg Usage    : 4.8x
Avg Fitness  : 4.6/5
Male %      : 82%
Female %    : 18%
Partnered % : 57%
Single %    : 42%
```

```
In [ ]: #----- Customer Profiles -----
profile = df.groupby('Product')[cont_cols].agg(['mean', 'median']).round(0)
print(profile)
profile.to_csv('profiles.csv')
print('\nKP281: Beginners, low income/miles.')
print('KP481: Mid-level.')
print('KP781: Premium - high fitness/income.')
```

	Age		Education		Usage		Fitness		Income \
	mean	median	mean	median	mean	median	mean	median	
Product									
KP281	29.0	26.0	15.0	16.0	3.0	3.0	3.0	3.0	46418.0
KP481	29.0	26.0	15.0	16.0	3.0	3.0	3.0	3.0	48974.0
KP781	29.0	27.0	17.0	18.0	5.0	5.0	5.0	5.0	75442.0

	Miles		
	median	mean	median
Product			
KP281	46617.0	83.0	85.0
KP481	49460.0	88.0	85.0
KP781	76568.0	167.0	160.0

KP281: Beginners, low income/miles.

KP481: Mid-level.

KP781: Premium – high fitness/income.

/tmp/ipykernel_524/1955129367.py:1: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

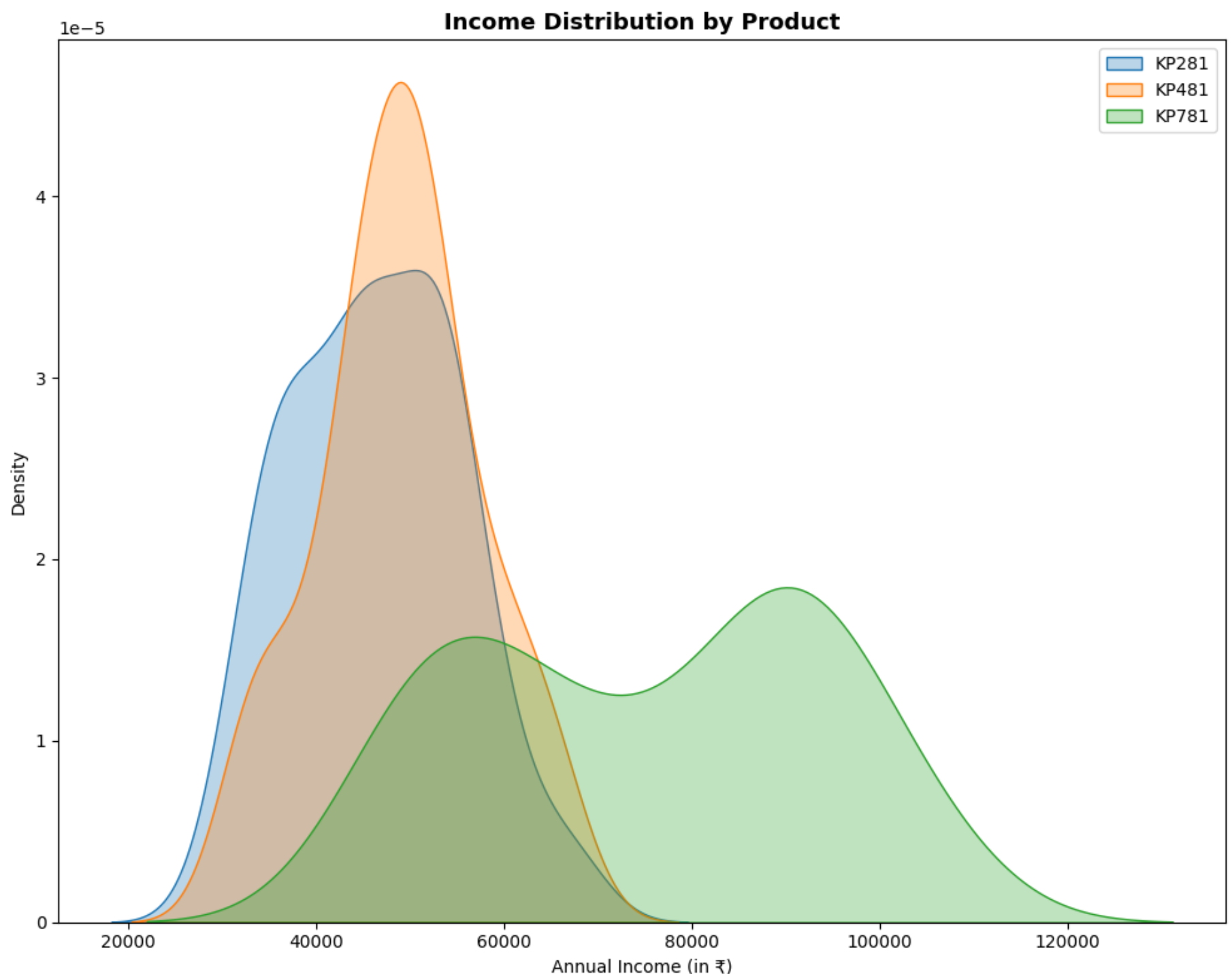
```
profile = df.groupby('Product')[cont_cols].agg(['mean', 'median']).round(0)
```

```
In [ ]: plt.figure(figsize=(10,8))

for product in ['KP281', 'KP481', 'KP781']:
    item = df[df['Product'] == product]
    sns.kdeplot(data = item['Income'], fill = True, alpha = 0.3 )

plt.title(f'Income Distribution by product')
plt.xlabel('Income')

plt.title('Income Distribution by Product', fontweight='bold', fontsize=13)
plt.xlabel('Annual Income (in ₹)')
plt.ylabel('Density')
plt.legend(['KP281', 'KP481', 'KP781'])
plt.tight_layout()
plt.show()
```



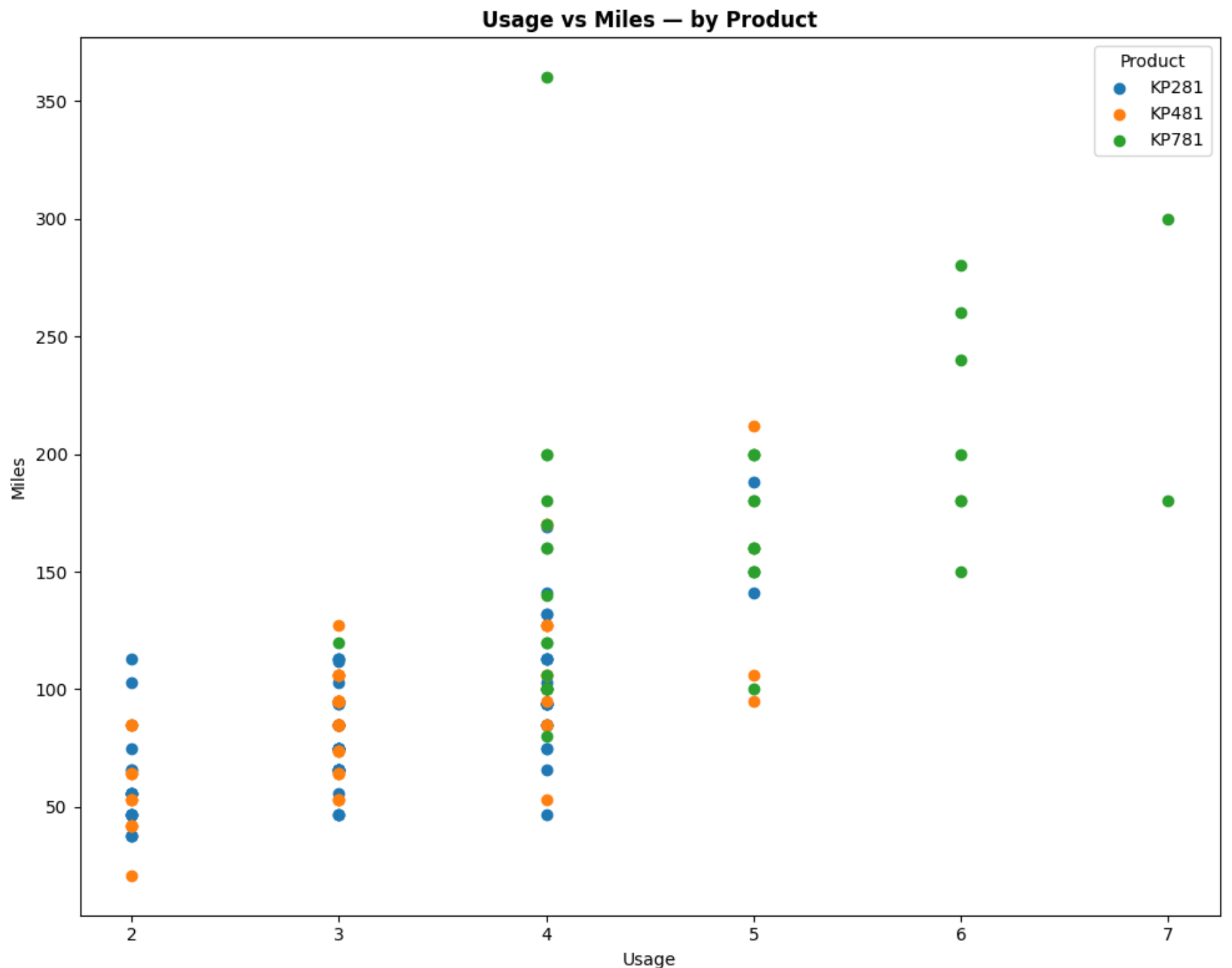
```
In [ ]: plt.figure(figsize= (10,8))
```

```

for product in ['KP281', 'KP481', 'KP781']:
    item = df[df['Product']== product]
    plt.scatter(item['Usage'], item['Miles'], label = product)

plt.title('Usage vs Miles – by Product', fontweight='bold')
plt.xlabel('Usage')
plt.ylabel('Miles')
plt.legend(title='Product')
plt.tight_layout()
plt.show()

```



9. Business Insights & Recommendations

Key Findings

Insight

1. KP281 dominates at ~44% of all sales — it's the most popular model
2. Female customers are heavily concentrated in KP281 — a big upsell opportunity for KP481/KP781
3. KP781 buyers are mostly male, high-income (\$75K+), run 150+ miles/week, and rate themselves 4–5/5 on fitness
4. Partnered customers disproportionately choose KP481 & KP781
5. Usage & miles are the strongest predictors of which product a customer buys
6. Income and Miles are strongly correlated with each other and with the product tier

Recommendations

1. Bundle deals for KP281 — Add accessories (mat, water bottle, starter plan) to increase basket size without cutting price.
2. Target women for upsell — Create female-centric marketing (fitness goals, community) to push KP481/KP781 sales.
3. Premium channels for KP781 — Advertise on LinkedIn, running apps, and fitness communities. Offer EMI / financing.
4. Couples/family packages — Partnered customers lean premium; promote 'family fitness' bundles.
5. Train staff on usage questions — Customers planning 4+ sessions or 100+ miles rarely stay at KP281; guide them up.
6. In-store fitness quiz — A quick 3-question quiz (fitness score, weekly miles goal, budget) can auto-recommend the right

product.